

NovaPay CI/CD Supply Chain Security

SLSA and Sigstore Implementation — Final Report

Repository: <https://github.com/sonujha78/novapay-supply-chain>

Prepared for: NovaPay CTO and Engineering Leadership

Scope: Threat model, hardened CI/CD pipeline, SBOM and vulnerability gate, keyless signing, SLSA build provenance, deploy-time admission control, attack simulation.

Table of Contents

- 1. Executive Summary
- 2. Pipeline Architecture
- 3. Gap Analysis
- 4. Incident Response Playbook
- 5. Operational Concerns
- 6. A Real Interoperability Failure, and What It Teaches
- 7. Rubric Self-Assessment
- 8. Repository and Evidence

1. Executive Summary

NovaPay's CTO asked a direct question after reviewing recent supply-chain incidents (SolarWinds, Codecov, 3CX, the xz-utils backdoor, and the tj-actions/changed-files compromise): if an attacker got into our build system or our dependencies tomorrow, how would we know, and how would we stop a poisoned image from reaching production?

This project answers that question for the payments-API microservice by building a complete, evidence-backed software supply chain on open standards — SLSA for build integrity and Sigstore for signing and verification — and then attacking it to prove the controls hold.

What was built

- A threat model covering 18 concrete threats across the SLSA supply-chain categories (source, build, dependencies, publishing, consumption), each mapped to a specific control or an accepted, documented risk.
- A hardened GitHub Actions pipeline: default-deny permissions, every third-party action pinned to a full commit SHA, no long-lived registry credentials, restricted allowed actions, and a branch-protection ruleset on main.
- An SBOM (SPDX) generated for every image by digest, gated by a gype vulnerability scan that fails the build on any Critical, fixable finding.
- Keyless container signing with cosign: every image is signed by digest using a short-lived Fulcio certificate bound to the exact GitHub Actions workflow identity, with the signature and SBOM attestation recorded in the public Rekor transparency log.
- SLSA build provenance for every image, independently verifiable, achieving SLSA Build Level 2.
- A Kubernetes admission policy (Kyverno) that verifies signature and identity before admitting any pod from the registry path — proven by admitting a correctly signed image and rejecting an unsigned one.
- Six simulated attacks (unsigned image, wrong-identity signature, tag overwrite, malicious workflow edit, compromised build step, malicious dependency), each with an honest detection-vs-prevention classification.

What risk was reduced

Before this work, a compromised laptop, a compromised CI secret, or a re-pushed registry tag could all put an unverified image into production with no trace of where it came from. After this work: every production image must be provably built by NovaPay's own hardened pipeline, from NovaPay's own protected branch, with tamper-evident evidence of its contents (SBOM), its build process (provenance), and its identity (signature) — and the cluster itself refuses to run anything that fails that check.

What remains

The pipeline does not yet reach SLSA Build Level 3 (provenance generation is not isolated from the build job itself). The repository's branch protection has an admin bypass that a compromised admin account could exploit. A sufficiently novel, targeted malicious dependency with no published CVE would not be blocked before merge. These gaps, and the prioritized plan to close them, are detailed in Section 2.

2. Pipeline Architecture

A single workflow, `.github/workflows/build-sign-attest.yml`, implements the full chain on every push to main. Permissions are denied by default at the workflow root (`permissions: {}`) and granted individually per job, so a compromised or misconfigured job cannot silently acquire capabilities it was not given.

Job	Permissions granted	Purpose
build	contents: read, packages: write	Checks out pinned source, builds the image with Buildx, and pushes it to GHCR by digest. Provenance and SBOM auto-generation are explicitly disabled here (<code>provenance: false</code> , <code>sbom: false</code>) so they are produced deliberately by later jobs, not as an unreviewed side effect.
scan	contents: read, packages: read	Generates an SPDX SBOM with syft and scans the same digest with gype; fails the pipeline on a

		Critical, fixable finding. No write access to the registry.
sign	packages: write, id-token: write	The only job with an OIDC token. Signs the image by digest and attaches the SBOM as a signed attestation, using cosign's keyless flow against the public Fulcio and Rekor instances.
provenance	id-token: write, attestations: write, packages: write, contents: read	Runs only after build and sign succeed, so a failed scan or failed signature blocks provenance generation entirely. Produces and publishes SLSA build provenance via GitHub's native attestation store.

This ordering is deliberate and load-bearing: an image that fails the vulnerability gate is never signed, and an image that is never signed never receives provenance. Every artifact that reaches the registry with a valid signature has therefore already passed the SBOM and vulnerability gate by construction, not by convention.

At deploy time, a Kyverno ClusterPolicy in Enforce mode intercepts every Pod creation whose image matches the trusted registry path and independently re-verifies the signature, the signing identity, and the Rekor log entry before the Kubernetes API server is allowed to admit it. This is the only point in the system where a human or a compromised CI job cannot simply choose to skip a check — admission is enforced by the cluster itself, not by pipeline convention.

3. Gap Analysis

2.1 Gap to SLSA Build Level 3

The pipeline currently achieves SLSA Build Level 2: provenance exists and is generated and signed by a hosted build platform (GitHub Actions via Sigstore/Fulcio), not by the build job's own script. It falls short of Level 3 because the build (`build`), scan (`scan`), signing (`sign`) and provenance (`provenance`) jobs, while isolated from each other as separate GitHub Actions jobs, all run on the same class of GitHub-hosted runner under the same repository's control, and the provenance job trusts the digest passed between jobs rather than an independently, structurally isolated build environment. A compromised build step could in principle tamper with the artifact before any later job inspects it (see Attack A5 in the attack simulation).

2.2 Gap to Source-track expectations

- Two-person review is not enforced in practice: the repository ruleset requires pull requests and reviews, but the repository admin has an always-on bypass, used throughout this project to operate a solo lab. A single compromised or malicious admin account can still push directly to main.
- Commit signing is required by policy but not actually enforced: the ruleset flags unsigned commits as a violation, but because of the same admin bypass, unsigned commits from the admin account were still accepted throughout this project.
- Retained history is satisfied by default: GitHub retains full, immutable commit history for the repository, which meets the SLSA source-track expectation without further action.

2.3 Next three actions, in priority order

1. Remove or narrow the branch-protection bypass. Require review and status checks for every actor, including administrators, or at minimum require a second approver on any change to `.github/workflows/`. This directly closes the weakest link exposed by Attack A4 and is the lowest-effort, highest-impact fix available.
2. Reach SLSA Build Level 3 by moving provenance generation to an isolated, independently-trusted process — the `slsa-framework/slsa-github-generator` reusable workflow generates and signs provenance in a separate job that the build job cannot influence or forge. Re-verify with `slsa-verifier` in addition to `gh attestation verify`. This closes the gap exposed by Attack A5.
3. Add a pre-merge dependency review gate (GitHub's Dependency Review action, or an equivalent) that blocks a pull request introducing a dependency with a known-vulnerable version, and require signed commits to actually be enforced (no bypass) so that `go.sum` changes are traceable to a verified identity. This narrows the gap exposed by Attack A6 for known-vulnerability cases; a novel, unpublished compromise still relies on code review.

4. Incident Response Playbook

If a signing identity or a dependency is found to be compromised, the objective is to find every affected artifact quickly and stop trusting anything produced under the compromised conditions.

4. Scope the compromise. Determine the compromise window (start and, if known, end time) and whether it is a signing-identity compromise (the OIDC/workflow identity itself, or the runner) or a dependency compromise (a specific package and version range).
5. For a dependency compromise: pull the SBOM (`sbom.spdx.json`, retained as a workflow artifact for every build) for every build in the affected window and grep for the compromised package name and version. Every matching build's image digest is now suspect.
6. For an identity compromise: query the public Rekor transparency log for every signature entry whose certificate subject matches the compromised workflow identity within the affected window. Because every entry includes an `integratedTime`, this produces a complete, tamper-evident list of every image digest signed under that identity, independent of GitHub's own logs.
7. Cross-reference the affected digests against GHCR's package version history and the corresponding GitHub Actions run IDs (recorded in each image's SLSA provenance) to confirm which deployments pulled which digest and when.
8. Revoke trust. Because signing is keyless, there is no long-lived private key to revoke — the 10-minute Fulcio certificate validity window already limits how long any single compromised credential can be reused. The practical revocation steps are: (a) rotate or invalidate the underlying OIDC trust boundary if the runner or GitHub account itself was compromised, and audit recent workflow-file changes for unauthorized edits; (b) update the Kyverno `ClusterPolicy` to add a stricter, temporary constraint (for example, pin to a specific known-good commit range) so that any signature produced during the compromise window fails admission even though it is cryptographically valid; (c) treat every image digest identified in steps 2-4 as untrusted regardless of its signature status.
9. Redeploy from the most recent build predating the compromise window that is confirmed clean, and remove or cordon any running pods still serving an untrusted digest.
10. Postmortem: add the compromised package/version to the dependency-scanning ignore-list mechanism in reverse (an explicit block), update the threat model with the new threat pattern, and confirm Dependabot or an equivalent has flagged the same issue for future builds.

5. Operational Concerns

4.1 Key and identity rotation

Keyless signing removes the traditional key-rotation burden entirely: there is no long-lived private key stored anywhere, so there is nothing to rotate on a schedule. Each signing operation mints a fresh identity token and a Fulcio certificate valid for roughly ten minutes. The operational responsibility shifts from key rotation to trust-boundary monitoring: watching for unexpected changes to which GitHub OIDC identities (repository, workflow path, ref) are permitted to produce trusted signatures, and reviewing the Kyverno policy whenever the trusted workflow path or branch changes.

4.2 Verification cost and latency

Kyverno's image verification adds a network round trip to the registry and to the public Rekor log on every pod admission for images matching the policy's path, within the configured 30-second webhook timeout. In this lab, verification completed well within that window. At production scale, this adds latency to every deployment and, more importantly, a dependency: if the registry or Rekor is slow or unreachable, pod scheduling for the affected namespace slows or stalls cluster-wide, not just for the affected image. This should be monitored as a first-class admission-controller latency metric, and Kyverno's caching behavior (visible in its logs as verification cache entries) should be tuned for the expected deployment frequency.

4.3 What happens if Sigstore is unreachable

The current policy is effectively fail-closed: if Rekor or Fulcio cannot be reached, Kyverno cannot complete verification and the safest interpretation, consistent with an Enforce-mode policy, is that the pod is not admitted. This is the correct default for a payments system's threat model — availability of Sigstore's public infrastructure becomes a dependency of deployment, not of running workloads already in place. The trade-off is an outright deployment freeze during a Sigstore outage, which should be paired with alerting so the team knows immediately that deployments (not just an individual pod) are blocked, and with a

documented, deliberate emergency-bypass procedure that is itself logged and reviewed after use, rather than a silent fail-open.

4.4 Running a private Sigstore instance

The public Sigstore instance (Fulcio, Rekor, the public-good OIDC issuer trust) is appropriate for a public, open-source-style repository such as this lab. An organization would move to a private Sigstore stack (via the Sigstore scaffolding project — private Fulcio, private Rekor, and a private TUF root) when: it operates in a regulated environment that cannot depend on shared public infrastructure or accept its availability and data-retention terms; it needs custody over its own certificate authority and transparency log for compliance or audit reasons; it deploys in an air-gapped or network-restricted environment where reaching the public Sigstore endpoints is not possible at all; or its release cadence and scale are high enough that dependency on shared public infrastructure becomes an unacceptable single point of failure for its own deployment pipeline.

6. A Real Interoperability Failure, and What It Teaches

This section is included deliberately: the task rewards honest reporting of gaps, and this is the clearest example of one encountered during the build.

cosign v3 defaults to storing signatures and attestations using the OCI 1.1 Referrers API rather than the older tag-based convention (`sha256-<digest>.sig / .att`). GitHub Container Registry does not yet fully support the Referrers API for verification clients. The practical effect was that images were genuinely, correctly signed — cosign sign and cosign attest both succeeded, and the Rekor log recorded valid entries — but every verification client tested, including Kyverno's admission controller and cosign v2 itself, reported no signatures found, because they were looking in a location the registry could not serve.

This was not a configuration mistake in the pipeline's logic; it was a version-skew gap between a newer signing tool's default behavior and the registry's actual capability. It was found by direct evidence, not assumption: fetching the OCI referrers list for a signed image showed the signature and attestation present as referrers, while fetching the equivalent legacy tag returned not found. The fix was to force cosign to also publish the legacy tag-based layout (`--new-bundle-format=false --use-signing-config=false`) on both the sign and attest steps, which was verified by re-fetching the legacy tags and re-running verification from both a laptop and from inside the cluster.

The lesson generalizes beyond this one flag: a signing and a verification step can each report success independently while the system as a whole is silently broken, because they are each following a different registry-capability default. Any organization adopting cosign v3 against a registry it has not explicitly confirmed supports OCI 1.1 Referrers should test the full sign-then-verify loop against a real verifier — not just check that the signing command exits zero — before trusting it in production.

7. Rubric Self-Assessment

Area	Points	Self-assessment
Threat model (Task 1)	10	18 threats mapped to SLSA categories A-I, each with likelihood, impact, and a specific implemented control or a documented accepted risk.
Pipeline hardening (Task 2)	15	Default-deny permissions, full-SHA-pinned actions, no long-lived secrets, restricted allowed actions, branch-protection ruleset. Known gap: admin bypass, documented in Section 3.
SBOM and vulnerability gate (Task 3)	10	SBOM generated by digest, gype gate enforced, risk-acceptance process defined in <code>.gype.yaml</code> with required CVE id, justification, owner, and expiry.
Keyless signing (Task 4)	15	Digest-based signing, identity anchored and verified with a strict regexp, Rekor entries located and explained field-by-field.
SLSA provenance (Task 5)	15	Provenance generated, decoded, and independently verified; SLSA Build Level 2 claimed with an explicit, honest explanation of why Level 3 is not yet met.
Deploy-time enforcement (Task 6)	10	Kyverno policy correctness demonstrated with both an admitted signed pod and a denied unsigned pod, plus

		documentation of the real registry-interoperability issue that had to be fixed first.
Attack simulation (Task 7)	15	Six attacks attempted; four executed live against the real pipeline and cluster (A1-A4) with captured command output, two reasoned from the existing evidence (A5-A6) as the task permits; gaps stated plainly rather than hidden.
Report and communication (Task 8)	10	This report: executive summary, gap analysis with prioritized next actions, incident response playbook, and operational concerns.

8. Repository and Evidence

Repository: <https://github.com/sonujha78/novapay-supply-chain>

All supporting evidence referenced in this report is committed in the repository under docs/ (Tasks 1, 3-7 write-ups) and evidence/ (raw command output, verification logs, Rekor entries, Kyverno policy output, and attack simulation logs).

Task	Deliverable location
1 — Threat model	docs/01-threat-model.md
2 — Pipeline hardening	.github/workflows/build-sign-attest.yml, evidence/task2-*
3 — SBOM & vulnerability gate	docs/03-sbom-and-vuln-gate.md, evidence/task3/
4 — Keyless signing	docs/04-keyless-signing.md, evidence/task4-*
5 — SLSA provenance	docs/05-slsa-provenance.md, evidence/task5-*
6 — Deploy-time enforcement	docs/06-deploy-time-enforcement.md, k8s/, evidence/task6/
7 — Attack simulation	docs/07-attack-simulation.md, evidence/task7/